SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU-574146

MINI PROJECT REPORT

ON

Intrusion Detection System

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN

CYBER SECURITY & CYBER FORENSICS

Submitted By,

Harsha Vardhan R 01SU23CF009
Sharana Basavana Gouda 01SU23CB026

Under the guidance of,

Mr. Yogesh Bandewar

IBM lecturer

Dept. of CSCF

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU-574146

Department of Cyber Security & Cyber Forensics

CERTIFICATE

This is to certify that the Technical Seminar entitled ***"Intrusion Detection System"*** is a bonafide work carried out by ***Harsha Vardhan R, Sharana Basavana Gouda*** bearing the **USN** ***01SU23CF009, 01SU23CB026*** in the partial fulfillment for the award of **Bachelor of Technology** in **Cyber Security & Cyber Forensics** of the **Srinivas University- Institute of Engineering & Technology** during the year **2025-26.** It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The mini project report has been approved as it meets the academic criteria outlined for the said degree.

Name & Signature

Mr. Yogesh Bandewar
(Project Coordinator)

Name & Signature

Prof. Gaurav Prasad
(HOD, Cyber security)

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU-574146

Department of Cyber Security & Cyber Forensics

DECLARATION

We , ***Harsha Vardhan R, Sharana Basavana Gouda*** the student of 6th semester, **Bachelor of Technology** in **Cyber Security & Cyber Forensics** of the **Srinivas University- Institute of Engineering & Technology**, Mukka, hereby declare that the mini project report entitled ***"Intrusion Detection System"*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology** in **Cyber Security & Cyber Forensics of Srinivas University-Institute of Engineering & Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: 20-03-2026
Place: Mukka

ABSTRACT

Intrusion attacks have become one of the major cybersecurity threats in modern computer networks. These attacks attempt to gain unauthorized access to systems, compromise sensitive data, or disrupt normal network operations. With the rapid growth of internet connectivity and digital services, protecting networks from malicious activities has become increasingly important. Traditional security mechanisms such as firewalls alone are not sufficient to detect sophisticated cyberattacks, making Intrusion Detection Systems an essential component of modern cybersecurity infrastructure.

This project presents the development of an **Intrusion Detection System (IDS)** designed to monitor and analyze network activities to identify suspicious or malicious behavior. The system works by examining network traffic and system logs to detect potential intrusions using a combination of rule-based analysis and machine learning techniques. By continuously monitoring incoming and outgoing network data, the system can detect abnormal patterns that may indicate an attack.

The proposed system analyzes network packets and compares them with known attack signatures while also applying anomaly detection methods to identify unusual activities. A machine learning model based on **Logistic Regression** is used to classify network behavior as either normal or malicious. The model is trained using a dataset containing examples of both legitimate and attack traffic, enabling the system to improve detection accuracy and reduce false alarms.

The application provides a simple and user-friendly interface that allows users to monitor network activity and receive alerts when suspicious behavior is detected. The system also includes visualization of detected threats and generation of reports for further analysis and documentation.

This project demonstrates how machine learning techniques and cybersecurity concepts can be combined to build an effective **Intrusion Detection System** capable of identifying potential threats and helping protect computer networks from unauthorized access and malicious attacks.

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of my mini project.

First and foremost, My sincere thanks go to **Mr. Yogesh Bandewar ,** Mini Project Coordinator, for coordinating the project process, offering timely suggestions, and being a constant source of motivation and assistance.

I am immensely grateful to **Prof. Gaurav Prasad** , Head of the Department of Cyber security , for his constant encouragement and valuable support throughout the course of my studies.

I would also like to extend my heartfelt thanks to **Dr. Ramakrishna N Hegde**, Dean, SUIET for his guidance and for facilitating the smooth conduct of the Mini Project Demonstration.

I am deeply thankful to the Management of **Srinivas University Institute of Engineering and Technology** for providing me with the opportunity to undertake this project as part of the academic curriculum. Their vision and commitment to practical learning have made this experience possible.

I would also like to acknowledge the support and encouragement of all the faculty members of the Department of Cyber Security, whose teachings laid the foundation for this practical learning.

Last but not least, I thank my family and friends for their continuous moral support and encouragement during the entire period of the project and report preparation.

This Mini Project has been a truly enriching experience and a vital step in my professional development.

CONTENTS

Chapter 1	**INTRODUCTION**	**7**
Chapter 2	**PROBLEM STATEMENT**	8
Chapter 3	**OBJECTIVES OF THE PROJECT**	**9**
Chapter 4	**SYSTEM ARCHITECTURE**	**10**
Chapter 5	**TECHNOLOGIES USED**	**11**
Chapter 6	**METHODOLOGY**	**12**
Chapter 7	**IMPLEMENTATION**	**13-19**
Chapter 8	**RESULT**	**20-22**
Chapter 9	**ADVANTAGES & LIMITATIONS**	**23**
Chapter 10	**FUTURE ENHANCEMENTS**	**24**
Chapter 11	**CONCLUSION**	**25**
Chapter 12	**REFERENCES**	**26**

LIST OF FIGURE

Chapter 8	List Of Figure	Page No.
8.1	CLIENT TERMINAL	20
8.2	SERVER TERMINAL	20
8.3	LOGIN PAGE	21
8.4	ADMIN DASHBOARD	21
8.5	MEMBER/USER CHATROOM	22

Chapter-1

INTRODUCTION

In the modern digital era, computer networks are widely used for communication, online transactions, cloud computing, and data sharing. While these technologies improve efficiency and connectivity, they also expose systems to various cybersecurity threats. Unauthorized access, data theft, malware attacks, and network intrusions are becoming increasingly common as cybercriminals continuously develop new techniques to exploit vulnerabilities in computer networks.

An intrusion occurs when an unauthorized user attempts to access or manipulate a computer system or network. These attacks may include activities such as hacking, denial-of-service attacks, malware injection, or unauthorized data access. Such intrusions can lead to serious consequences including financial loss, data breaches, and disruption of critical services.

Traditional security mechanisms such as firewalls and antivirus software provide a first level of protection by filtering traffic and blocking known threats. However, these security tools are not always capable of detecting sophisticated or previously unknown attacks. Many cyberattacks bypass traditional security systems by exploiting new vulnerabilities or by disguising malicious activities as normal network behavior.
To overcome these limitations, **Intrusion Detection Systems (IDS)** are used to monitor and analyze network traffic and system activities in order to identify suspicious behavior. An IDS continuously observes network data and generates alerts when potential intrusions or abnormal activities are detected. These systems help administrators respond quickly to threats and reduce the risk of damage to the system.

Intrusion Detection Systems generally use two main approaches: **signature-based detection**, which identifies attacks by comparing network traffic with known attack patterns, and **anomaly-based detection**, which detects unusual behavior that deviates from normal network activity. By combining these techniques, IDS can effectively identify both known and unknown threats.

The objective of this project is to design and develop an **Intrusion Detection System** capable of monitoring network activities and detecting possible cyberattacks. The system analyses network traffic and classifies activities as either normal or malicious using rule-based methods and machine learning techniques.
This project demonstrates how cybersecurity principles and machine learning can be applied to build a practical system that enhances network security and helps protect computer systems from unauthorized access and malicious activities.

Chapter 2

PROBLEM STATEMENT

With the rapid growth of computer networks and internet-based services, organizations and individuals increasingly rely on digital systems to store and process important information. However, this widespread connectivity has also created opportunities for cyber attackers to exploit network vulnerabilities and gain unauthorized access to systems. Intrusion attacks such as hacking attempts, malware infections, denial-of-service attacks, and unauthorized data access have become common threats that can cause serious damage to computer networks.

Traditional security mechanisms such as firewalls and antivirus software provide a basic level of protection by filtering traffic and blocking known malicious activities. However, these systems are often limited in their ability to detect advanced or previously unknown attacks. Many cyberattacks are designed to bypass traditional defenses by disguising malicious traffic as normal network activity, making them difficult to detect using conventional methods.

Another challenge is that many existing intrusion detection solutions are designed for large-scale enterprise environments. These systems may require expensive infrastructure, complex configurations, and specialized technical knowledge to operate effectively. As a result, smaller organizations, educational institutions, and individual users may find it difficult to implement and maintain such systems.
Additionally, modern cyber threats are constantly evolving, and attackers frequently modify their techniques to avoid detection. Systems that rely only on predefined attack signatures may fail to identify new or unknown intrusion patterns. This limitation highlights the need for more intelligent detection mechanisms that can recognize abnormal behavior in network traffic.

Therefore, there is a need for an effective intrusion detection system that can monitor network activities, identify suspicious behavior, and alert users to potential security threats. The system should be capable of analysing network data using both rule-based detection and machine learning techniques to improve accuracy and adaptability.

This project aims to address these challenges by developing an Intrusion Detection System that can detect potential intrusions by analysing network traffic and identifying abnormal patterns. The system demonstrates how machine learning and cybersecurity concepts can be applied to enhance network security and help protect systems from unauthorized access and malicious activities.

Chapter 3

OBJECTIVES OF THE PROJECT

The main objective of this project is to develop an efficient and reliable **Intrusion Detection System** that can monitor network activities and identify potential security threats. With the increasing number of cyberattacks targeting computer networks, it has become essential to develop systems that can detect unauthorized access and suspicious behavior before significant damage occurs. This project focuses on creating a system capable of analyzing network traffic and identifying activities that may indicate an intrusion.

One of the primary goals of the project is to design and implement a system that continuously monitors network traffic and system activities. By analyzing incoming and outgoing data packets, the system can identify patterns that are commonly associated with malicious activities such as unauthorized access attempts, abnormal traffic patterns, or suspicious user behavior. This monitoring process helps in detecting possible security breaches at an early stage.

Another important objective of the project is to apply **machine learning techniques** to improve the detection capability of the system. Instead of relying only on predefined rules or attack signatures, the system uses a machine learning model trained on a dataset containing examples of both normal and malicious network activities. This allows the system to learn patterns from historical data and classify network behavior as either normal or suspicious with improved accuracy.

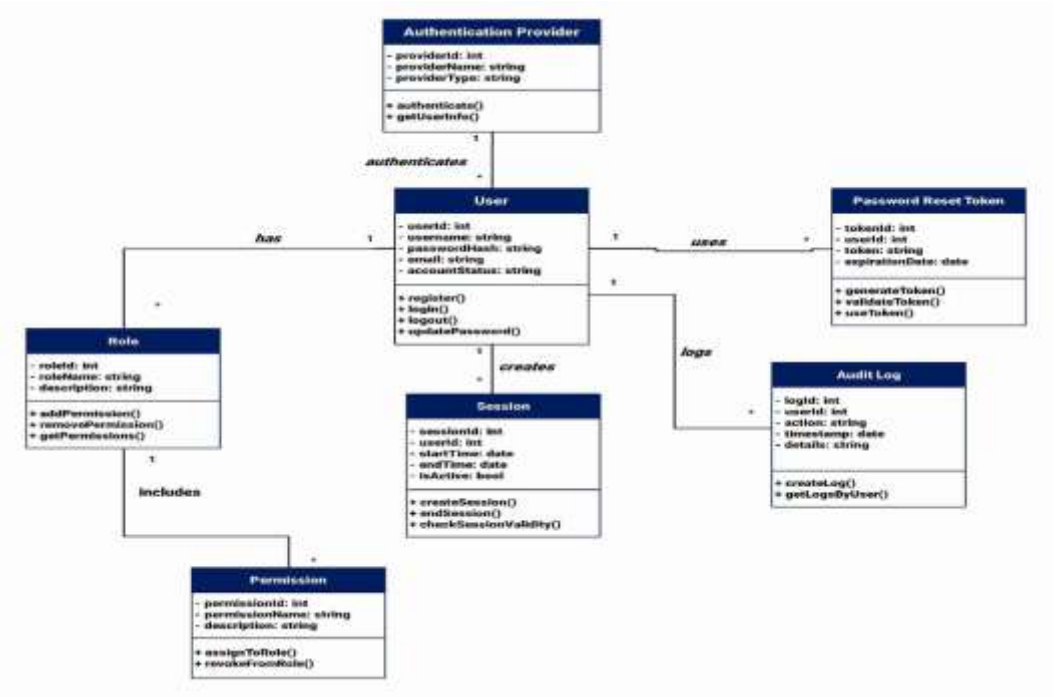

FIG 3.1: Class Diagram

Chapter-4

SYSTEM ARCHITECTURE

The **Intrusion Detection System** is designed using a multi-layer architecture where each layer performs a specific task in monitoring and analysing network activities. This layered structure improves the accuracy and reliability of the detection process by combining multiple analysis techniques. The system continuously observes network traffic and processes the collected data through different detection stages to identify suspicious or malicious behavior.

When network traffic enters the system, it is first processed by the **Data Collection Layer**. This layer is responsible for capturing network packets and system activity logs from the network environment. The collected data includes information such as source IP address, destination IP address, protocol type, packet size, and connection duration. This information forms the basis for further analysis within the system.

The next stage is the **Preprocessing and Feature Extraction Layer**. In this layer, the raw network data is cleaned and transformed into a structured format suitable for analysis. Important features related to network behavior are extracted, such as traffic frequency, connection attempts, and protocol usage. These features help the system understand normal network patterns and identify unusual behavior.

After preprocessing, the data is passed to the **Detection Layer**, which performs the core analysis of the system. This layer uses two main approaches: rule-based detection and machine learning classification. The rule-based detection component compares network activities with known attack patterns or predefined security rules. If a match is found, the system identifies the activity as a potential intrusion.

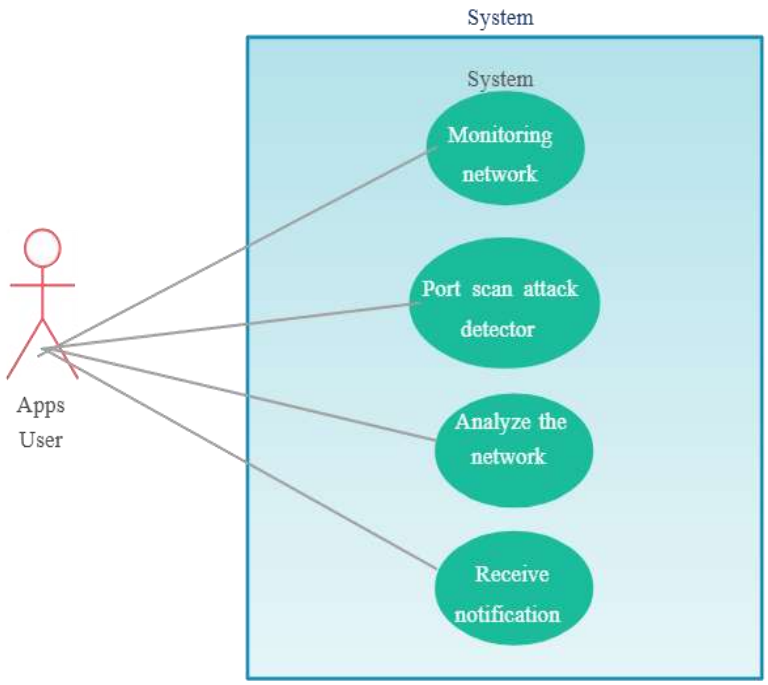

FIG 4.1: Use-case Diagram

Chapter-5

TECHNOLOGIES USED

The **Intrusion Detection System (IDS)** is developed using modern web technologies that support frontend interface development, backend processing, database management, and security implementation. These technologies work together to create a secure and efficient system for monitoring and managing application activities.

Frontend

• **React.js (with Vite)** – React.js is used to build a dynamic and responsive user interface for the application. Vite is used as a development tool that provides a fast environment for building and running the React application.

• **JavaScript** – JavaScript is used to implement the functionality and logic of the frontend components and enables communication with the backend services.

Backend

• **Node.js** – Node.js is used to create the server environment that processes client requests and manages communication between the frontend and the database.

• **Express.js** – Express.js is a lightweight framework used with Node.js to create APIs, manage routing, and handle HTTP requests and responses within the system.

Database

• **MongoDB** – MongoDB is used as the database to store application data such as user information, login records, and security events. It provides a flexible and efficient way to manage large amounts of data.

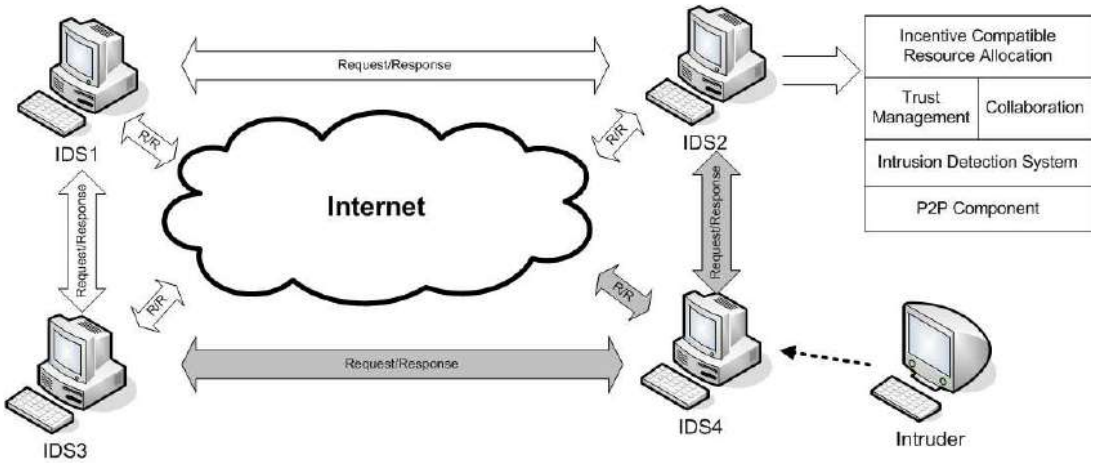

FIG 5.1: Collaboration diagram

Chapter-6

METHODOLOGY

The **Intrusion Detection System (IDS)** is designed using a client–server architecture that integrates a frontend interface, a backend server, and a database. The system works by monitoring user activities and managing authentication while implementing security mechanisms to prevent unauthorized access. The methodology focuses on secure communication, data processing, and controlled access to the system.

The process begins with the **user interacting with the frontend interface**, which is developed using **React.js**. The frontend provides pages for user login, registration, and interaction with the system. When a user performs an action such as logging in or accessing protected resources, the request is sent from the frontend to the backend server.

The **backend server**, developed using **Node.js and Express.js**, processes incoming requests from the client. It handles the application logic, user authentication, and communication with the database. The backend verifies user credentials and ensures that only authorized users can access restricted parts of the system.

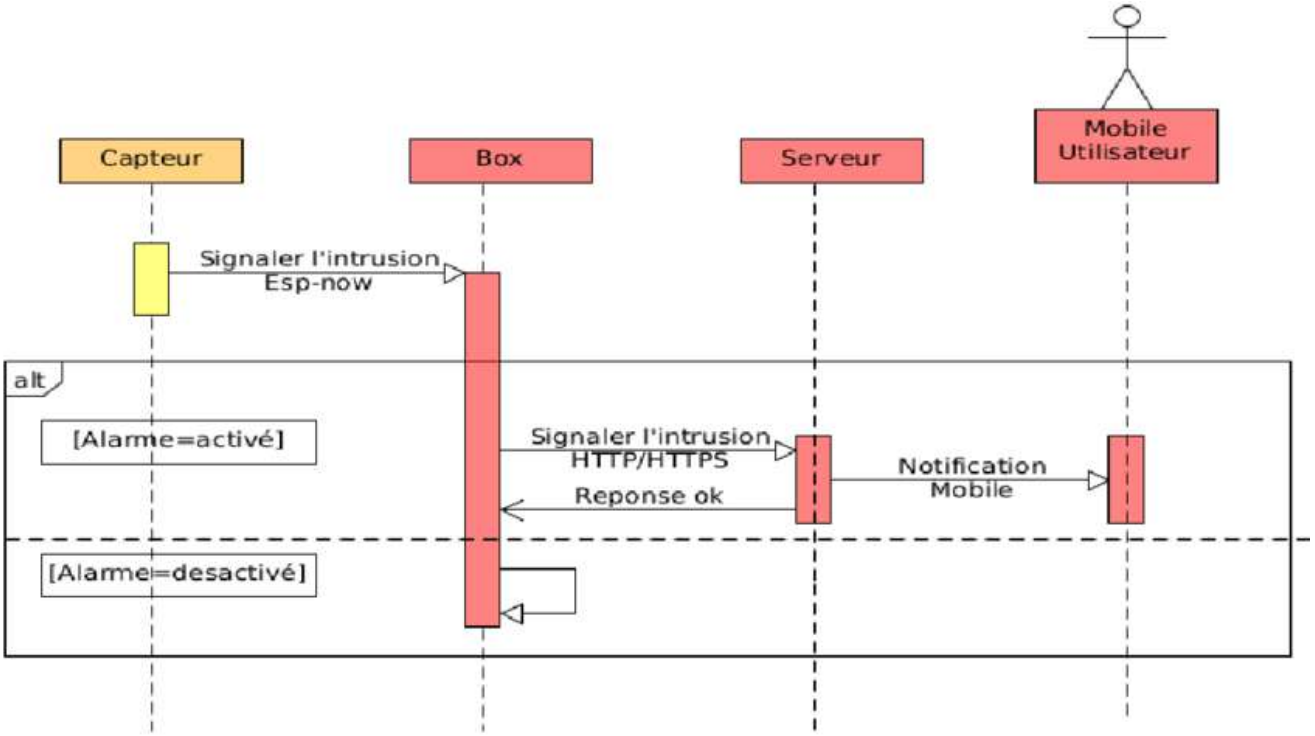

FIG 6.1 : Sequence diagram

CHAPTER 7

IMPLEMENTATION

7.1 Introduction

The Intrusion Detection System is implemented using **Node.js** and **Express.js**. The system provides secure authentication through user registration and login features. Security techniques such as password hashing, token-based authentication, and login rate limiting are used to protect the system from unauthorized access. The implementation uses database models such as **User** and **SecurityEvent** to store user information and record security-related activities.

7.2 Libraries Used

The following libraries are used in the implementation:

- **bcryptjs** – Used to hash and verify passwords.
- **jsonwebtoken** – Used to generate authentication tokens.
- **express-rate-limit** – Prevents excessive login attempts.
- **MongoDB** – Stores user and security event data.

7.3 Registration Implementation

The **/register** route allows new users to create an account. The system first validates the input fields such as username, email, and password. It also verifies the email format and ensures that the password length is at least six characters.

The system checks whether the email or username already exists in the database. If the user is valid, a new account is created and stored in the database. The first registered user is assigned the **admin role**, while other users receive the **member role**.

After successful registration, a **JWT token** is generated and a security event is logged.

7.4 Login Implementation

The **/login** route authenticates users. A rate limiter restricts the number of login attempts to **10 attempts within 15 minutes** to prevent brute-force attacks.

During login, the system checks the email and password. The entered password is compared with the stored hashed password using bcrypt. If the credentials are valid, a JWT token is generated and returned to the user. If the login fails, the system records a **LOGIN_FAILED** security event. Successful logins are recorded as **LOGIN_SUCCESS**.

7.5 Security Event Logging

The system logs important security events using the SecurityEvent model. These events include:

- User registration
- Failed login attempts
- Successful login attempts

Each event stores information such as the user ID, event type, severity level, and IP address. This helps in monitoring suspicious activities and detecting possible intrusion attempts.

7.6 Summary

The implementation uses secure authentication techniques including password hashing, JWT authentication, and login rate limiting. Security events are recorded to track user activities and detect potential threats. These features help improve the overall security of the system.

7.7Implementation Code

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const rateLimit = require("express-rate-limit");

const User = require("../models/User");
const SecurityEvent = require("../models/SecurityEvent");

const router = express.Router();

// Rate limiter for login attempts
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,
  message: { message: "Too many login attempts. Try again later." },
});

// Function to get JWT secret
function getJwtSecret() {
```

```
  const secret = process.env.JWT_SECRET || process.env.JWTSECRET;
  if (!secret) {
    throw new Error("Missing JWT secret in environment variables");
  }
  return secret;
}

// Function to generate token
function signToken(user) {
  return jwt.sign(
    { userId: user._id, role: user.role, username: user.username },
    getJwtSecret(),
    { expiresIn: "1d" }
  );
}

/* --------------------------
   USER REGISTRATION
---------------------------*/

router.post("/register", async (req, res) => {
  try {
    const { username, email, password } = req.body;

    // Validate input fields
    if (!username || !email || !password) {
      return res.status(400).json({
        message: "username, email, and password are required",
      });
    }

    // Validate email format
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
```

```
  return res.status(400).json({ message: "Invalid email format" });
}

// Password validation
if (password.length < 6) {
  return res.status(400).json({
    message: "Password must be at least 6 characters",
  });
}

// Check if user already exists
const existing = await User.findOne({
  $or: [{ email }, { username }],
});

if (existing) {
  return res.status(409).json({
    message: "Email or Username already exists",
  });
}

// Assign role (first user becomes admin)
const userCount = await User.countDocuments();
const role = userCount === 0 ? "admin" : "member";

// Create user
const user = await User.create({
  username,
  email,
  password,
  role,
  isActive: true,
});
```

```
    // Log security event
    await SecurityEvent.create({
      user: user._id,
      eventType: "REGISTER",
      severity: "low",
      ip: req.ip,
      details: { email },
    });

    // Generate token
    const token = signToken(user);

    res.status(201).json({
      message: "User registered successfully",
      token,
      user: {
        id: user._id,
        username: user.username,
        email: user.email,
        role: user.role,
      },
    });
  } catch (err) {
    res.status(500).json({
      message: "Register failed",
      error: err.message,
    });
  }
});

/* --------------------------
   USER LOGIN
---------------------------*/
```

```
router.post("/login", loginLimiter, async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({
        message: "All fields required",
      });
    }

    const user = await User.findOne({ email });

    if (!user || !user.isActive) {
      await SecurityEvent.create({
        eventType: "LOGIN_FAILED",
        severity: "medium",
        ip: req.ip,
        details: { email },
      });

      return res.status(401).json({
        message: "Invalid credentials",
      });
    }

    const validPassword = await bcrypt.compare(password, user.password);

    if (!validPassword) {
      await SecurityEvent.create({
        user: user._id,
        eventType: "LOGIN_FAILED",
        severity: "medium",
        ip: req.ip,
        details: { reason: "WRONG_PASSWORD" },
```

```
    });

    return res.status(401).json({
      message: "Invalid credentials",
    });
  }

  // Log successful login
  await SecurityEvent.create({
    user: user._id,
    eventType: "LOGIN_SUCCESS",
    severity: "low",
    ip: req.ip,
    details: {},
  });

  const token = signToken(user);

  res.json({
    message: "Login successful",
    token,
    role: user.role,
  });

 } catch (err) {
  res.status(500).json({
    message: "Login failed",
    error: err.message,
  });
 }
});

module.exports = router;
```

Chapter-8

RESULT

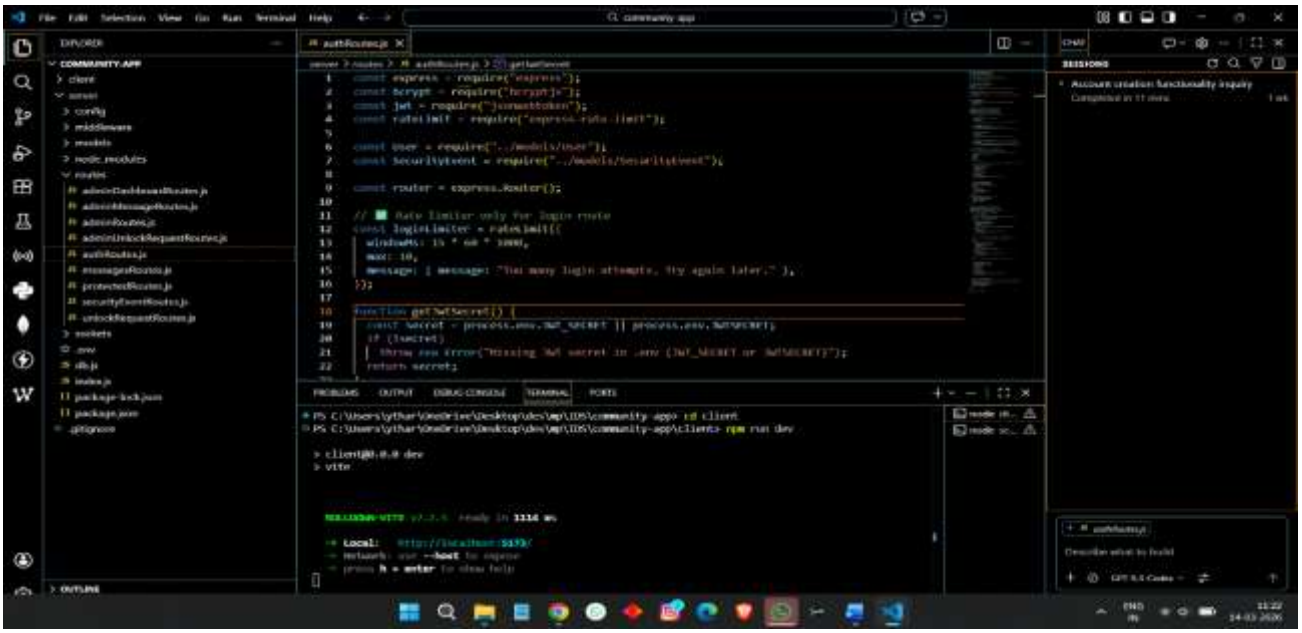

FIG 8.1 Client terminal

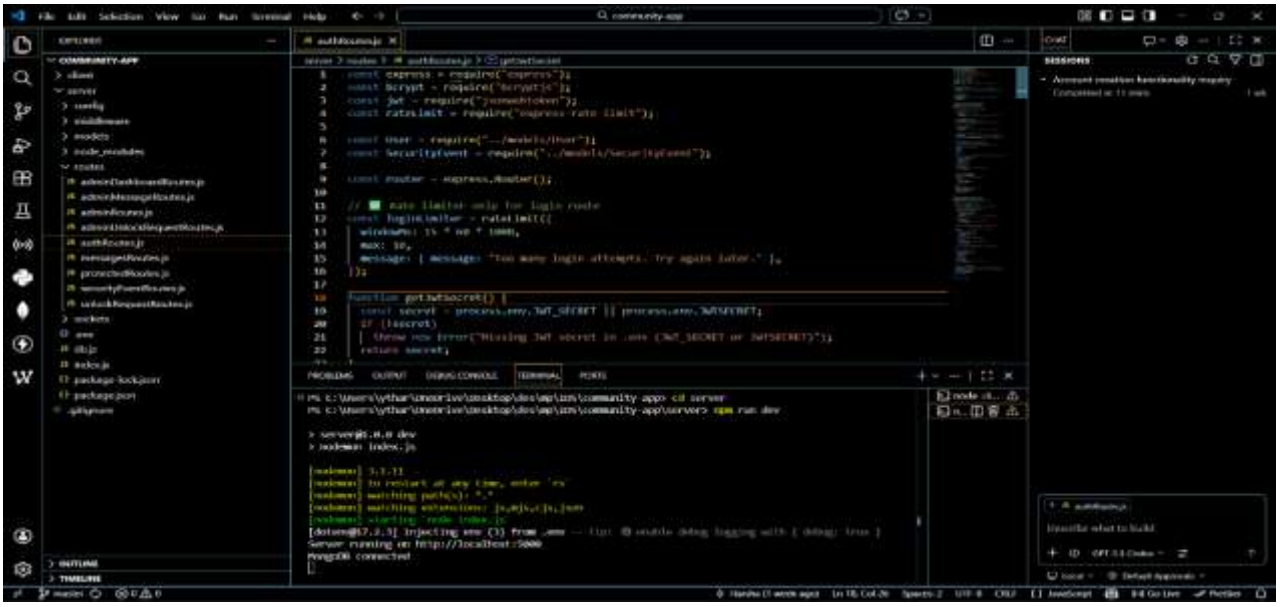

FIG 8.1 Server terminal

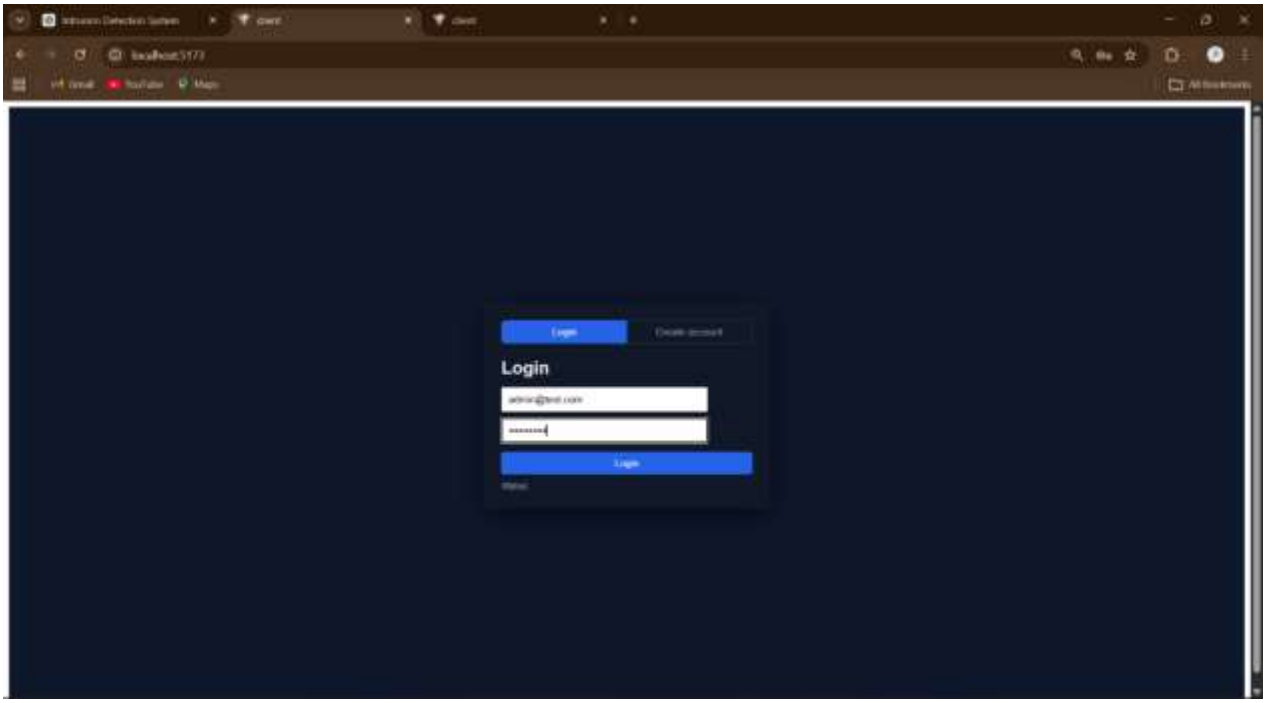

FIG 8.3 Login page

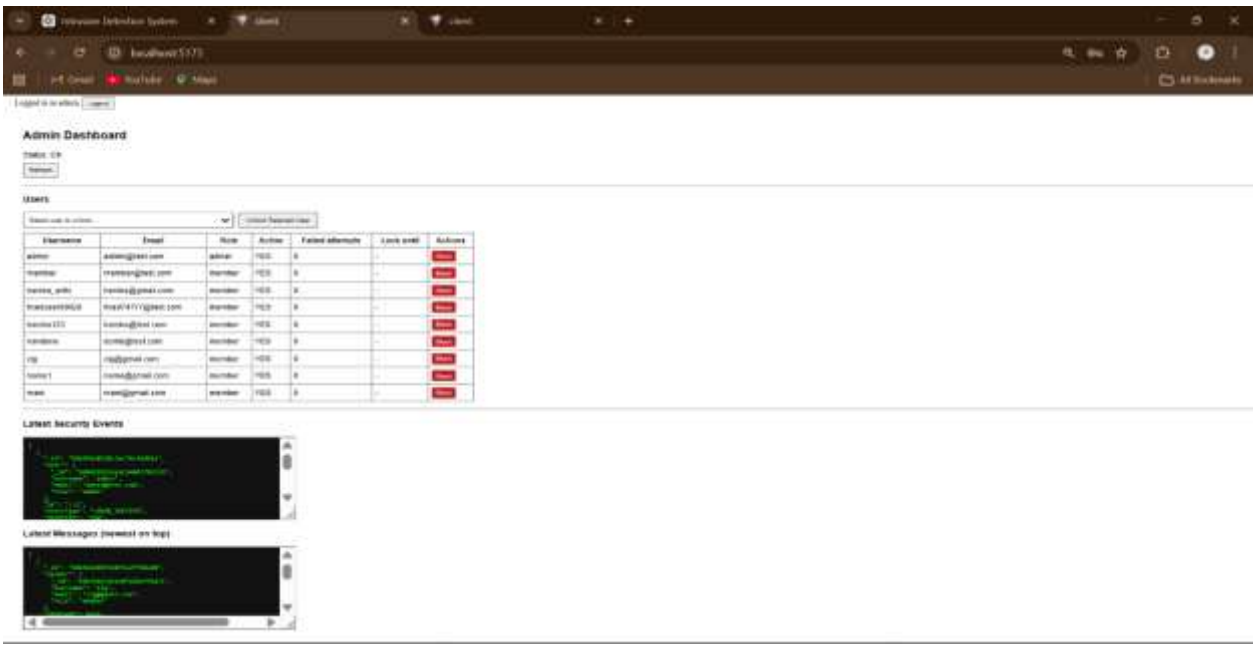

FIG 8.4 Admin Dashboard

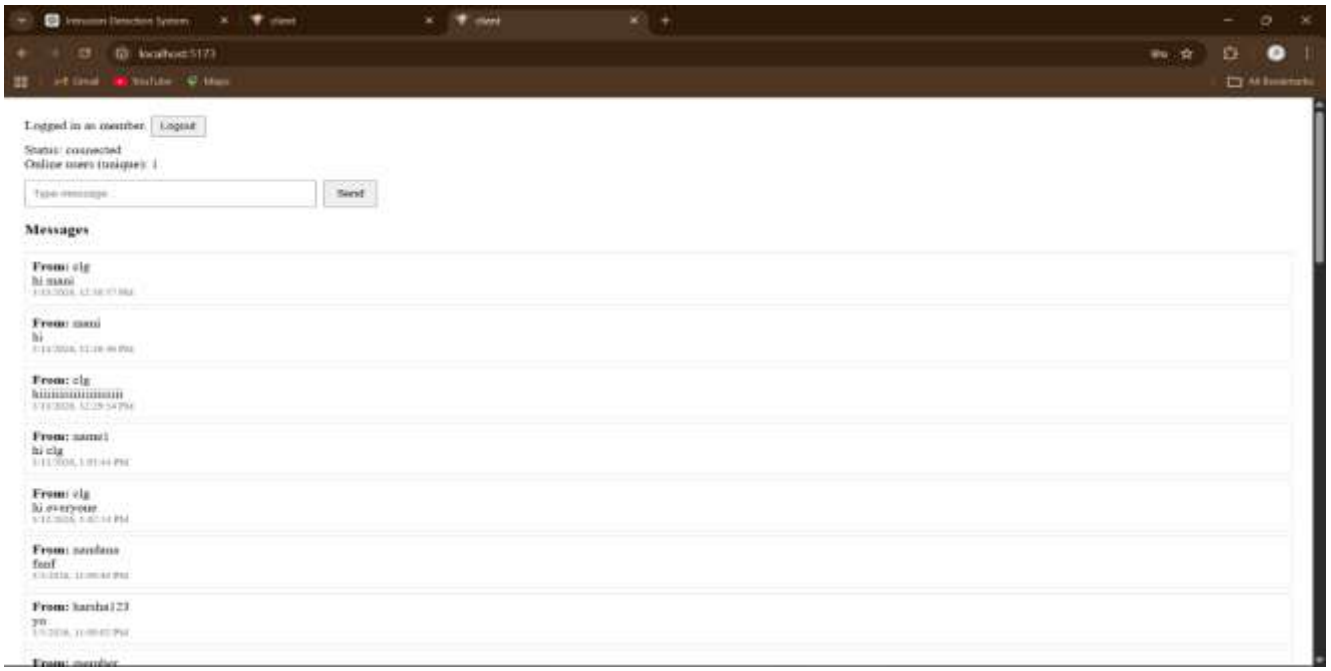

FIG 8.5 Member/user chatroom

Chapter-9

ADVANTAGES & LIMITATIONS

9.1 Advantages

1. **Improved Security**
   The system helps detect unauthorized login attempts and suspicious activities, improving the overall security of the application.
2. **Password Protection**
   User passwords are securely stored using **bcryptjs**, which protects user credentials from being exposed.
3. **Token-Based Authentication**
   The system uses **JSON Web Token** to provide secure authentication and maintain user sessions.
4. **Protection Against Brute Force Attacks**
   The login route uses **express-rate-limit** to limit the number of login attempts and prevent repeated login attacks.
5. **Security Event Logging**
   The system records important events such as registration, successful login, and failed login attempts, which helps monitor suspicious behavior.

9.2 Limitations

1. **Limited Attack Detection**
   The system mainly detects login-related security events and may not detect all types of cyber attacks.
2. **Dependence on Internet and Server**
   The application requires a running server and database to function properly.
3. **Basic Intrusion Detection**
   The system currently uses simple event logging and authentication checks instead of advanced intrusion detection algorithms.
4. **Scalability Issues**
   For very large systems with many users, additional security mechanisms and monitoring tools may be required.

Chapter-10

FUTURE ENHANCEMENT

The current Intrusion Detection System provides basic security features such as authentication, rate limiting, and security event logging. However, the system can be further improved by adding advanced features and technologies.

.1 Real-Time Intrusion Detection

Future versions of the system can include real-time monitoring of network traffic to detect suspicious activities instantly. This will allow the system administrator to respond quickly to potential security threats.

2 Advanced Machine Learning Techniques

Machine learning algorithms can be integrated to improve the accuracy of intrusion detection. By analyzing patterns in network traffic, the system can automatically identify abnormal behavior and detect new types of attacks.

3 Multi-Factor Authentication

An additional layer of security can be implemented using multi-factor authentication. Along with the password, users may be required to verify their identity using a one-time password or authentication application.

4 Security Dashboard

A graphical dashboard can be developed to display security events, login attempts, and system alerts. This will help administrators easily monitor system activities and detect suspicious behavior.

5 Integration with Monitoring Tools

The system can be integrated with professional security monitoring tools such as **Splunk** or **Wireshark** to provide deeper analysis of network traffic and security incidents.

Chapter-11

CONCLUSION

The Intrusion Detection System developed in this project aims to enhance the security of web applications by monitoring user authentication activities and detecting suspicious behavior. With the increasing number of cyber threats and unauthorized access attempts, implementing proper security mechanisms has become essential for protecting user data and system resources.

In this project, a secure authentication system was developed using **Node.js** and **Express.js**. These technologies provide an efficient and scalable environment for developing server-side applications. The system allows users to register and log in securely while ensuring that sensitive information such as passwords is properly protected.

To strengthen security, the system uses **bcryptjs** to hash user passwords before storing them in the database. This ensures that even if the database is compromised, the actual passwords cannot be easily accessed. Additionally, the system implements token-based authentication using **JSON Web Token**, which allows secure communication between the client and server. JWT tokens help maintain authenticated user sessions while reducing the risk of unauthorized access.

Another important feature implemented in this system is login rate limiting using **express-rate-limit**. This mechanism restricts the number of login attempts within a specific time period, helping to prevent brute-force attacks where attackers repeatedly try different password combinations to gain access.
The system also records security-related events such as user registration, successful login attempts, and failed login attempts. These events are stored in the database and can be analyzed to identify unusual patterns or suspicious activities. By maintaining such logs, administrators can monitor system behavior and take appropriate action when potential threats are detected.

Overall, the project successfully demonstrates how basic intrusion detection concepts can be integrated into a web application through secure authentication mechanisms and activity monitoring. The developed system provides a foundation for implementing more advanced intrusion detection techniques in the future, such as real-time network monitoring, machine learning-based threat detection, and advanced security analytics.
In conclusion, this project highlights the importance of implementing strong security measures in modern web applications. The proposed system improves application security, protects user credentials, and provides an effective way to monitor and detect possible intrusion attempts.

Chapter-12

REFERENCES

- **Node.js Official Documentation.**
  Available at: https://nodejs.org/en/docs
- **Express.js Official Guide.**
  Available at: https://expressjs.com
- **MongoDB Documentation.**
  Available at: https://www.mongodb.com/docs
- **bcryptjs NPM Package Documentation.**
  Available at: https://www.npmjs.com/package/bcryptjs
- **JSON Web Token Documentation.**
  Available at: https://jwt.io/introduction
- **express-rate-limit NPM Documentation.**
  Available at: https://www.npmjs.com/package/express-rate-limit
- **Github link:**
  Available at: https://github.com/harsha5870/community-app.git